The door that opens by itself

Lesson 5 · A motor that listens to your sensor



JA05 STEAM and Service Learning · 23 Oct 2026

Who cannot open a door?

Hands full

Shopping in both hands. No hand left for the handle.

Wheelchair

Pull the door, roll backwards, go through. Twice as hard.

Crutches

One hand holds you up. One hand is not enough.

Hospital

Touching handles moves germs from hand to hand.

An automatic door is not a luxury. For some people it is the difference between going in and going home.

Today you build one. Sensor sees a person · servo opens the door.

Today: 100 minutes



Steps 0 to 2 are read-and-run. Step 3 is the one you write. Step 4 (phone) only if you get there.

Every group leaves with a servo that moves. That already counts as finished.

CHECKPOINT By minute 45 every group has a servo that moves to three positions.

Get today's files

```
Terminal
MacBook:~ s20xxxxxx$ cd ~/JA05
MacBook:JA05 s20xxxxxx$ ./get_latest.sh
Checking GitHub for new files...
Done. You have the latest files.

New folder: 20261023_L5
```

1. Saves anything you changed into my_work
2. Gets the newest files from GitHub
3. Never touches your ESP32

CHECKPOINT A folder 20261023_L5 appears in JA05.

STUCK? permission denied? Type: bash get_latest.sh

Your own code from last lesson is on the ESP32, not in JA05. It is safe.

How to use the code today

1. Bottom-right of Thonny says **MicroPython (ESP32)**
2. **View > Files**, then open JA05 > 20261023_L5
3. Right-click the step file > **Upload to /**
4. Double-click it under **MicroPython device**
5. **Run (F5)** without changes first
6. Change only Try it / YOUR CODE lines. **Cmd+S** saves on the ESP32

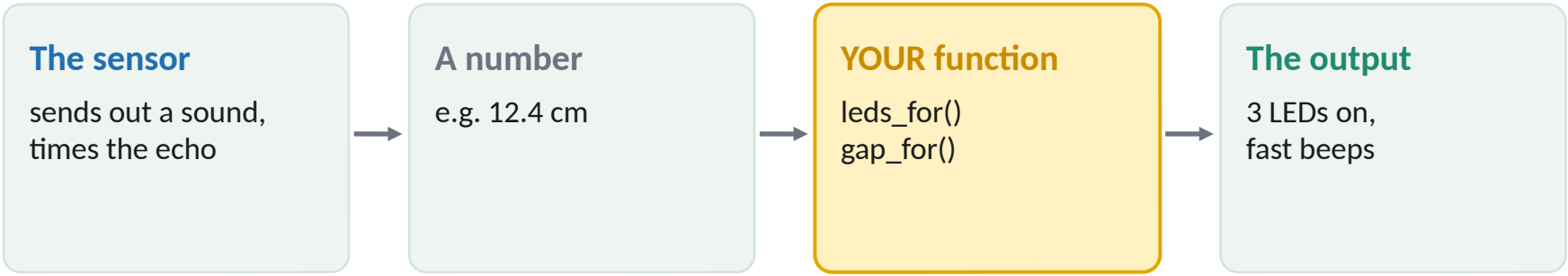
Don't type code from the slides

Slides show only PART of a program. The whole program is already in the file.

`step1_servo_sweep.py` · lines 20–21

This header tells you which file and which lines to look at.

Look back: what did you build?



The sensor and the LEDs are the easy half. The thinking happened in the middle box, and you wrote that part.

Tell your partner, 30 seconds each

What did your parking sensor do when a hand came close?

Then: what took you longest?

Wiring? The maths? Finding the file? Say which.

You have used functions all along

`print("Hello")`	Someone else wrote it	Shows text in the Shell
`time.sleep(1)`	Someone else wrote it	Waits one second
`led.value(1)`	Someone else wrote it	Turns the pin on
`leds_for(12)`	**YOU wrote it**	**Gives back 3**

again and
again, about
10 times a

The only new thing in Lesson 3 was this: **you made one of your own.**

The parts of a function

```
def leds_for(distance):
    return 5 - int(distance // 5)
```

- `def` = I am making one
- `leds_for` = the name you choose
- `distance` = what goes IN
- `return` = what comes OUT

Writing def does not run it.
It only tells Python the recipe exists.

leds_for(12) runs it.
distance becomes 12 inside, just for that one run.

Step 0: watch a function run

0

No wiring. Upload step0_functions.py, run it, and read the Shell from top to bottom.

step0_functions.py · lines 11–21

```
11  print("1. The program starts here")
12
13
14  def greet(name):
15      print("   Hello,", name)
16
17
18  print("2. greet is DEFINED. No hello yet.")
19
20  greet("Class 2A")
21  greet("Ms Chan")
```

CHECKPOINT Line 2 of the Shell appears BEFORE any hello.

STUCK? Nothing in the Shell? Press Run, not just save.

Terminal

```
1. The program starts here
2. greet is DEFINED. No hello yet.
   Hello, Class 2A
   Hello, Ms Chan
3. Two hellos: greet was CALLED
   twice.
4. distance -> how many LEDs
   12 cm -> 3 LEDs
```

Try it: delete the two greet(...) lines. The hellos stop, the rest still runs.

The order Python really does it

0

step0_functions.py · lines 11–23

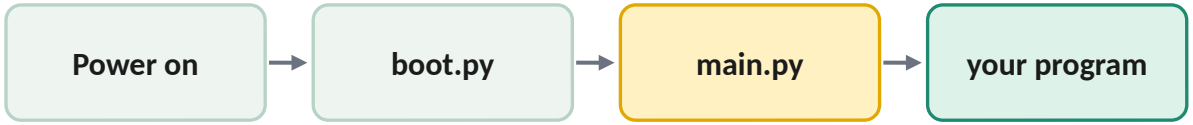
```
11  print("1. The program starts here")
12
13
14  def greet(name):
15      print("   Hello,", name)
16
17
18  print("2. greet is DEFINED. No hello yet.")
19
20  greet("Class 2A")
21  greet("Ms Chan")
22
23  print("3. Two hellos: greet was CALLED twice.")
```

1. **line 11** — prints “1. The program starts here”
2. **lines 14–15** — SKIPPED. Python only remembers that greet exists
3. **line 18** — prints “2. greet is DEFINED …”
4. **line 20** — calls greet, jumps UP to line 15, prints, comes back
5. **line 21** — calls greet again. Line 15 runs a second time
6. **line 23** — prints “3. Two hellos …”

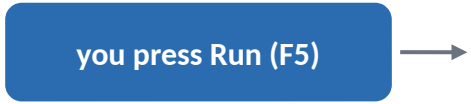
Python reads top to bottom, except when a call sends it somewhere else.

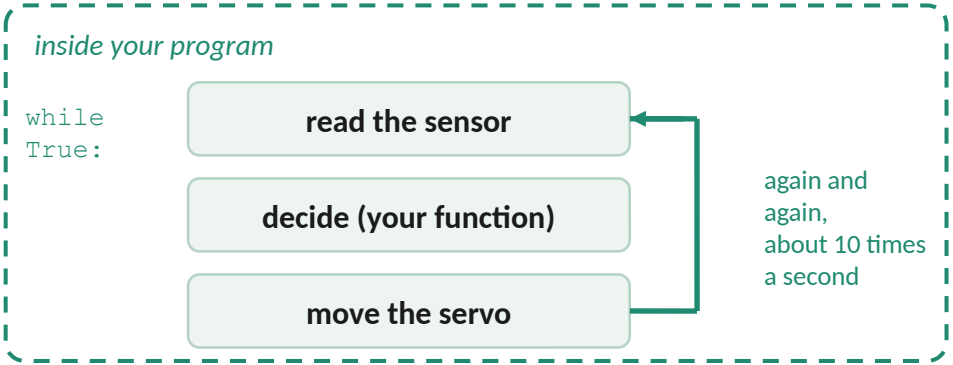
Line 15 never runs until something calls greet.

What happens when the ESP32 starts



or, while you are working:





Stop, or Ctrl+C breaks the loop. The finally: lines then tidy up.

The file lives on the chip. Unplug the Mac, use a power bank: it still runs.

main.py is the only name the ESP32 starts by itself. Any other name needs the Run button.

Trace it: what does leds_for(12) give?

0

step0_functions.py · lines 27–35

```
27  def leds_for(distance):
28      return max(0, min(5, 5 - int(distance // 5)))
    ⋮
35  answer = leds_for(12)
```

1. **distance becomes 12** — the number on line 35
2. **12 // 5 is 2** — how many whole 5 cm fit into 12
3. **5 - 2 is 3** — max/min keep it 0 to 5
4. **gives back 3** — line 35 puts 3 into answer

Now you. On paper, no computer.

- leds_for(40) gives back?
- leds_for(2) gives back?
- How many times does line 28 run when the for loop on line 32 goes round five times?
- Which line of your Lesson 3 program decided WHEN the LEDs change? Was it inside leds_for?
- gap_for(20) gave 0.4 s. What gives gap_for(10)?

Self-check: tick what you can do

☐	I can read a line like leds_for(12) and say what it gives back	*Step 0, slide before this*
☐	I can say the difference between writing def and calling the function	*Step 0: the Shell order*
☐	I can change a number inside a function and predict what happens	*Step 0 Try it lines*
☐	I can write the inside of a function when I am told what goes in and out	*that is today's Step 3*

Cannot tick one? Say so now. Three people here can explain it, and one of them is sitting next to you.

You have written this twice already

Lesson 3

`leds_for(distance)`

how many LEDs?

gives back: 0 to 5

Lesson 4

`gap_for(distance)`

how long to wait?

gives back: 0.1 to 0.5 s

Today

`angle_for(distance)`

which angle?

gives back: 0 or 90

The same shape every time

```
distance  ->  your function  ->  one decision
```

A number goes in. You decide what comes out. The rest of the program is already written.

Pick your track

Does your LED bar light up when your hand gets closer?

A — Not yet

- Wire the sensor: Lesson 3 PDF page 25
- Run 20261009_L3/step5_solution.py
- Then start Step 1 with everybody else

B — Yes, it works

- Start at Step 1: wire the servo
- Step 2: set_angle()
- Step 3: you write angle_for()

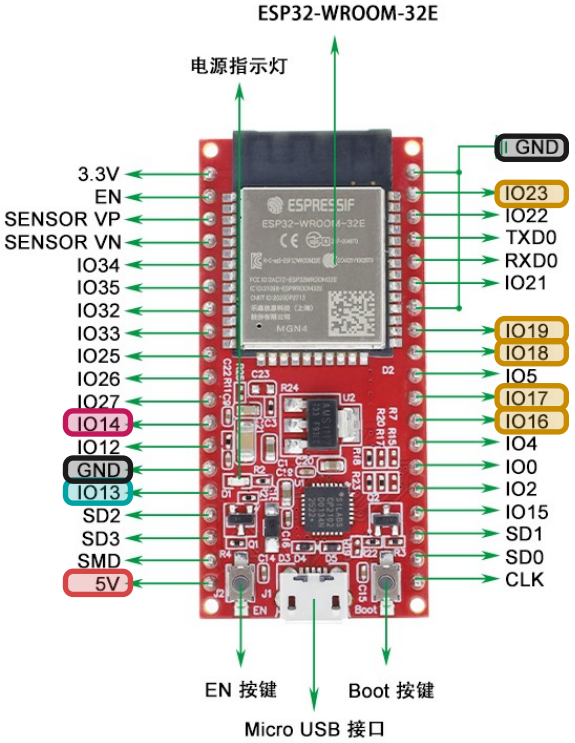
You do not need the buzzer today. The servo is the new part for everyone.

Track A: catch up fast

A

1. Upload the Lesson 3 files you need to the ESP32
2. Check the LEDs: step3a_check_leds.py
3. Add the sensor: Lesson 3 PDF, page 25
4. Run step4_ultrasonic_print.py, then step5_solution.py
5. LED bar works? Go straight to Step 1

CHECKPOINT Hand closer, more LEDs on.

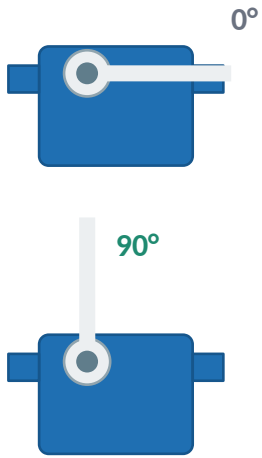


A servo is an output that moves

Part	You say	It does
LED	value(1)	light, on or off
Buzzer	value(1)	sound, on or off
Servo	**an angle: 0 to 180**	**turns there and HOLDS it**

It holds the angle. Let go of the door and it stays open.

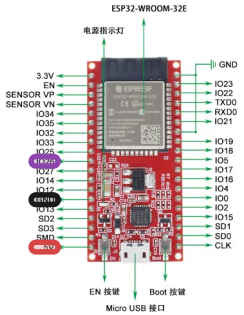
Inside: a small motor, gears, and a sensor that checks where the arm is, thousands of times a second.



Same servo. Two angles.

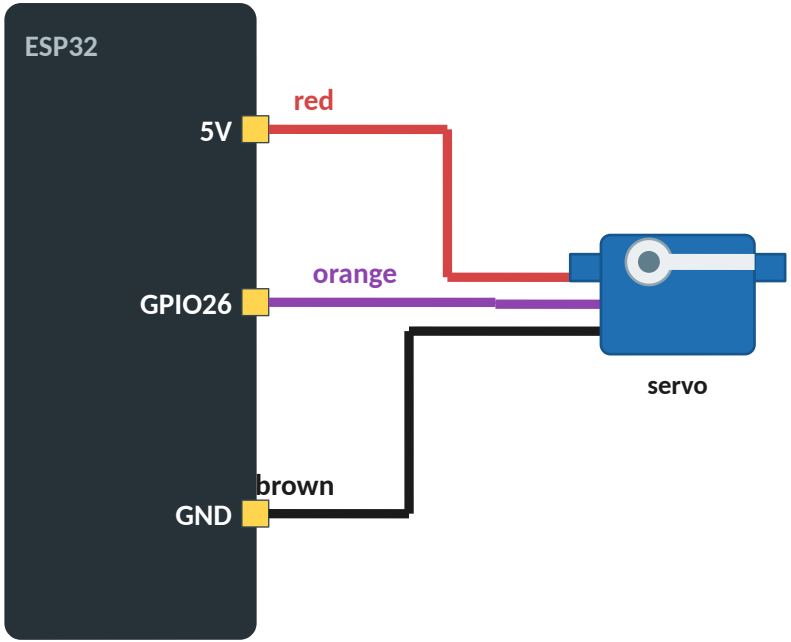
Wire the servo - read this first

Servo wire	Goes to
brown or black	GND
red	**5V (never 3V3)**
orange or yellow	GPIO26 (IO26)



IO26 is on the left side, one below IO25.

The sensor also needs 5V. Use the red rail on the breadboard so one pin feeds both.



Red goes to 5V. On 3V3 the ESP32 resets when the servo moves.

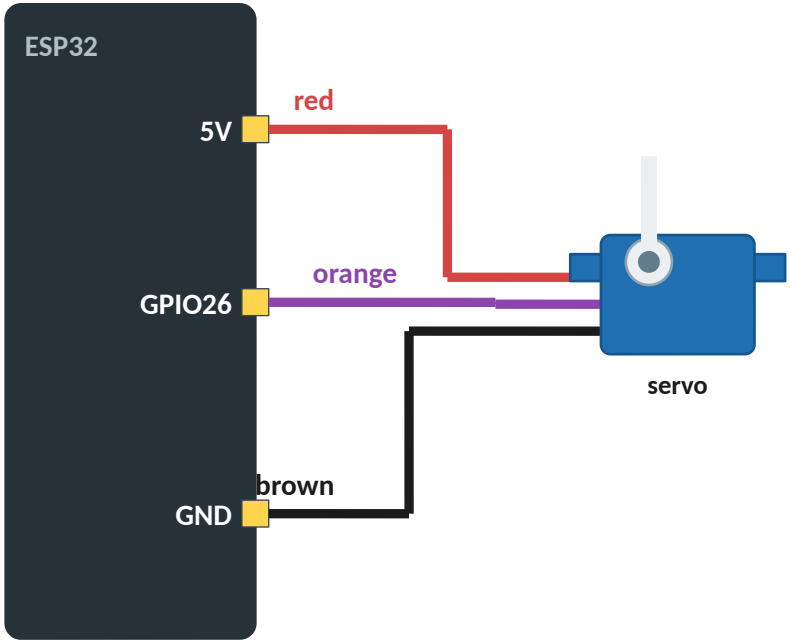
Step 1: make it move

1

Upload step1_servo_sweep.py and run it without changing anything.

step1_servo_sweep.py · lines 20–34

```
20  servo = PWM(Pin(26))
21  servo.freq(50)
    ⋮
29      servo.duty_u16(1638)
30      print("0 degrees")
31      time.sleep(1)
32
33      servo.duty_u16(4915)
34      print("90 degrees")
```



CHECKPOINT The horn jumps to three positions, then back.

STUCK? Nothing moves? Check orange on IO26, red on 5V.

Red goes to 5V. On 3V3 the ESP32 resets when the servo moves.

Read the code: Step 1

1

step1_servo_sweep.py · lines 17–26

```
17  from machine import Pin, PWM
18  import time
19
20  servo = PWM(Pin(26))
21  servo.freq(50)
22
23  #   angle    duty_u16
24  #     0       1638
25  #    90       4915
26  #   180       8100
```

Predict, then try

Which number would send it to 45 degrees? Try your guess on line 29.

Line 20 PWM switches GPIO26 on and off. Same tool as the LED brightness and the buzzer pitch.

Line 21 50 times a second. A servo only listens at this speed.

Line 23–26 The duty number that means each angle. Nobody remembers these.

Line 29 duty_u16(1638): go to 0 degrees and hold.

Step 2: say the angle, not the number

2

No rewiring. Upload step2_servo_angle.py and run it.

step2_servo_angle.py · lines 21–27

```
21  def set_angle(deg):
22      if deg < 0:
23          deg = 0
24      if deg > 180:
25          deg = 180
26      duty = MIN_DUTY + deg * STEP_PER_DEGREE
27      servo.duty_u16(int(duty))
```

step2_servo_angle.py · lines 32–34

```
32  print("angle:", angle)
33  set_angle(angle)
34  time.sleep(0.8)
```

CHECKPOINT It stops at every angle in your list.

Why bother?

Before

```
servo.duty_u16(4915)
```

After

```
set_angle(90)
```

Both do the same thing. Only one of them can be read by a human six weeks later.

STUCK? Buzzing and stuck? Free the horn.

Read the code: set_angle()

2

step2_servo_angle.py · lines 17–27

```
17  MIN_DUTY = 1638
18  STEP_PER_DEGREE = 36.4
19
20
21  def set_angle(deg):
22      if deg < 0:
23          deg = 0
24      if deg > 180:
25          deg = 180
26      duty = MIN_DUTY + deg * STEP_PER_DEGREE
27      servo.duty_u16(int(duty))
```

Work it out

- 1638 + 90 × 36.4 = ? Compare it with the table in Step 1.
- What does set_angle(500) do? Why does nothing break?

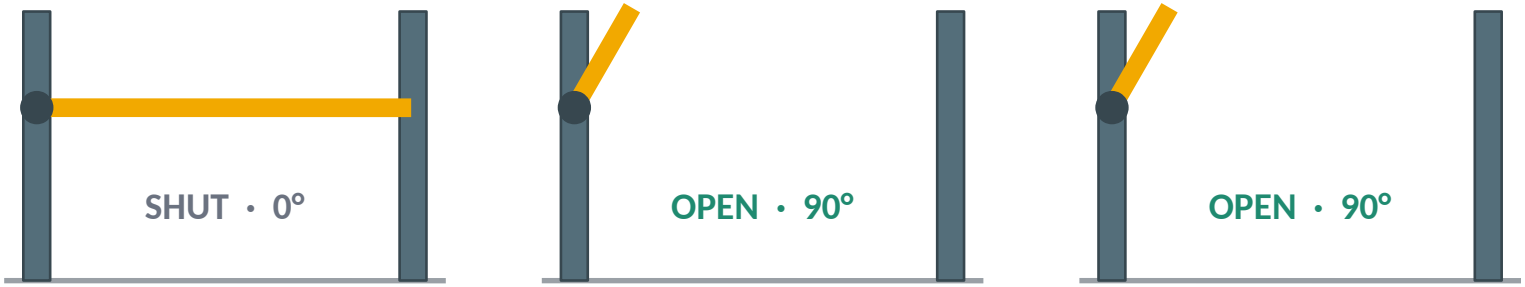
Line 17 1638 is 0 degrees: the smallest signal the servo accepts.

Line 18 Every degree adds 36.4. So 90 degrees adds about 3276.

Line 22–25 Guard rails: nothing below 0, nothing above 180.

Line 26–27 int() because duty_u16 only takes whole numbers.

How should the door behave?



Nobody there

Somebody within 15 cm

They passed: open 2 s more

approach → open → walk through → wait 2 s → shut

Think: why must the door wait before it shuts? What happens to a slow walker if it shuts the instant the sensor stops seeing them?

Step 3: you write angle_for()

3

step3_auto_gate.py · lines 32–35

```
32  OPEN_ANGLE = 90
33  CLOSED_ANGLE = 0
34  NEAR_CM = 15
35  HOLD_LOOPS = 20
```

step3_auto_gate.py · lines 71–82

```
71  def angle_for(distance):
    ⋮
76      # ----- YOUR CODE STARTS HERE -----
    ⋮
80      # Use OPEN_ANGLE and CLOSED_ANGLE, not 90 and 0.
81
82      return CLOSED_ANGLE                # replace this line
```

Replace line 82 only. Do not change any other line.

distance	angle_for() gives back
None (sensor saw nothing)	CLOSED_ANGLE
less than NEAR_CM	OPEN_ANGLE
anything else	CLOSED_ANGLE

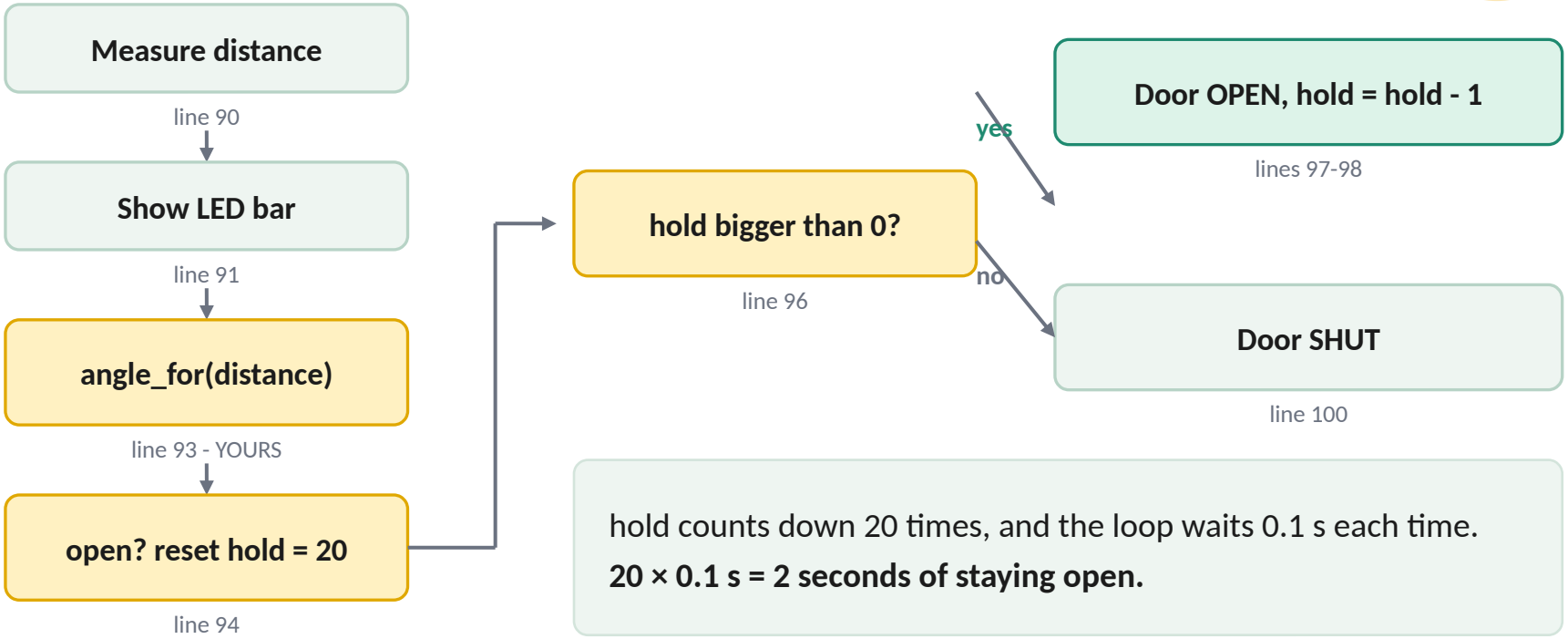
Watch out: the sensor gives None sometimes. Test for it FIRST or the program crashes.

CHECKPOINT Hand near: opens. Hand away: shuts 2 s later.

STUCK? Never opens? Line 82 is still CLOSED_ANGLE.

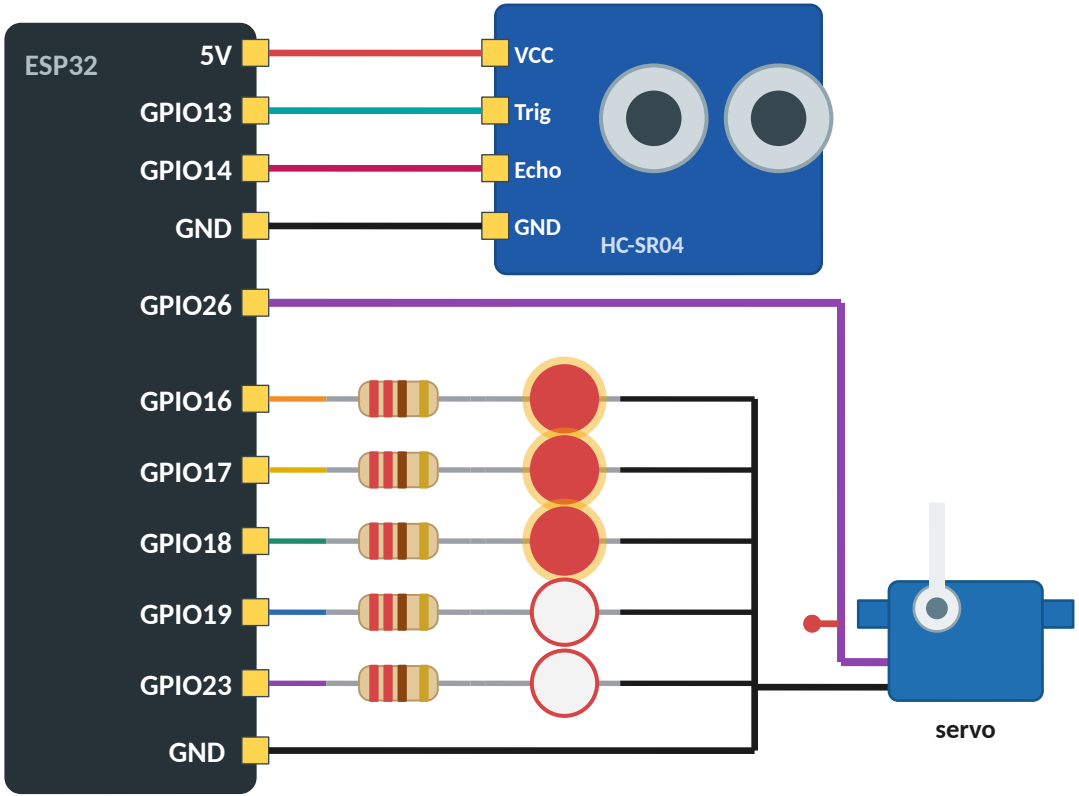
Read the code: what the loop does

3

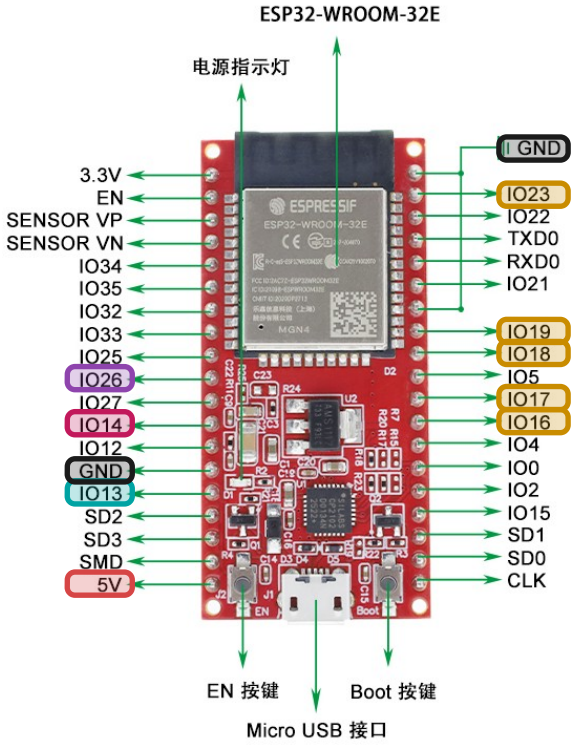


Your function is used in one place only: line 93.

The whole circuit



Dashed red: the servo's power wire goes to the breadboard's 5V rail, the same rail as the sensor.



Outlined: every pin used today. Purple = servo.

Step 4: open it from your phone

4

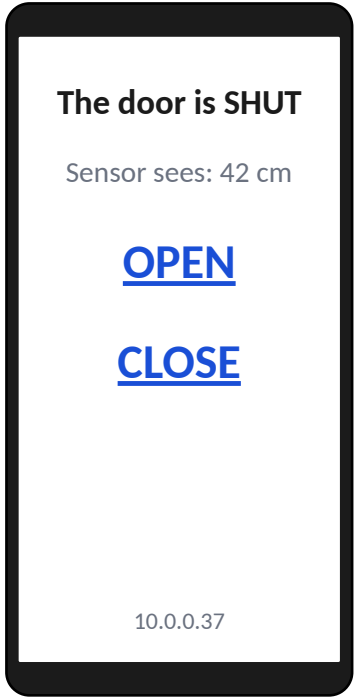
Your phone → class Wi-Fi → your ESP32 → the servo

Every board gets its own address, like 10.0.0.37. Yours is printed in the Shell when the program starts.

Terminal

```
Connecting to css_STEAM ...
Connected. Open this on your phone:  http://10.0.0.37/
```

Your phone must be on the class Wi-Fi. On mobile data the page will never load: the two are not on the same network.



Step 4: type the Wi-Fi name yourself

4

step4_web_gate.py · lines 16–17

```
16  SSID = ""
17  PASSWORD = ""
```

```
SSID = "css_STEAM"        PASSWORD =
"steam2026!"
```

step4_web_gate.py · lines 110–115

```
110  while True:
111      client, address = server.accept()
112      request = client.recv(1024).decode()
113
114      if "GET /open" in request:
115          is_open = True
```

1. Open your copy under **MicroPython device**
2. Type both lines exactly, keep the quotes
3. **Cmd+S** saves it on the ESP32, not in JA05
4. Run it. Read the IP in the Shell
5. Open that address on your phone. Press OPEN

CHECKPOINT The horn moves when you tap OPEN.

STUCK? Wi-Fi FAILED? Check both quotes are still there.

Who is your door for?

Sensor only

Opens for anybody who comes near. Nobody has to touch anything.

Phone only

Somebody decides who comes in. Good for a gate, bad for a shop.

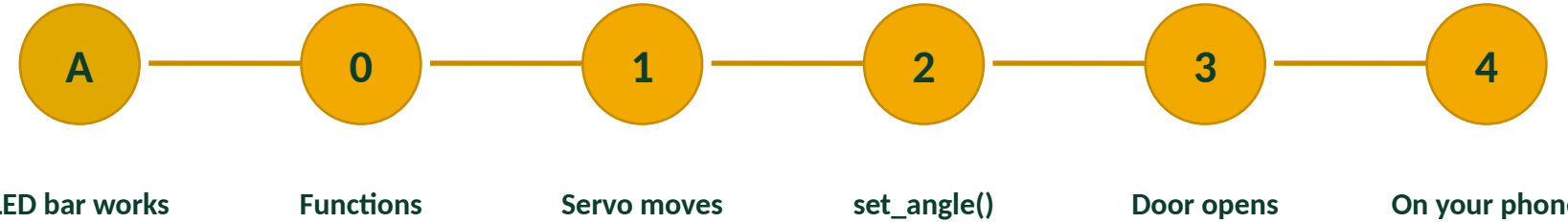
Both

Opens by itself, and a person can override it.

Talk to the person next to you: name one door in this school.

- Who struggles with it, and when?
- Would your sensor see them in time?
- What would go wrong at the busiest minute of the day?

What did you reach today?



- A servo is an output that holds a position. PWM meant brightness, then pitch, now angle.
- Third time: a function turns a distance into a decision. def makes it, the brackets run it.
- A door that shuts too fast is a door that hurts somebody. Hold-open time is part of the design.

Next: pick the real problem we build for.